



RESONANCIA

Informe ejecutivo de arquitectura y
cimientos técnicos

Casa del Cuenco

Fecha de corte: 22 de agosto de 2026

Versión del informe: 1.0

Propósito: facilitar la incorporación de programadores y la toma de decisiones técnicas.

1. Resumen ejecutivo

RESONANCIA es una plataforma digital de bienestar sonoro compuesta por una aplicación móvil nativa, una API central, una base de datos relacional, un panel administrativo web y varias bibliotecas compartidas. El producto está construido principalmente en **TypeScript** y se organiza como un **monorepo pnpm**, lo que permite compartir contratos, tipos y lógica entre los distintos componentes.

La aplicación móvil utiliza **React Native sobre Expo**, con proyectos nativos iOS y Android, navegación por archivos y módulos nativos para audio, animación, video, compras, notificaciones y sesiones en vivo. El backend utiliza **Node.js, Express y PostgreSQL**, con **Drizzle ORM** para el acceso a datos. La definición de la API vive en un contrato **OpenAPI**, desde el cual se generan clientes React Query y validadores Zod.

El audio no depende de un único reproductor. La arquitectura separa:

- reproducción de sesiones y control del sistema;
- mezcla de ambientes y capas;
- bucles sincronizados y sin separación audible;
- audio de descanso;
- audio de chat;
- coordinación para que los subsistemas no compitan.

La animación también está especializada. **React Native Reanimated**, Worklets, Gesture Handler y SVG forman la base de las escenas animadas y de **Geometrix**, el editor visual de composiciones geométricas.

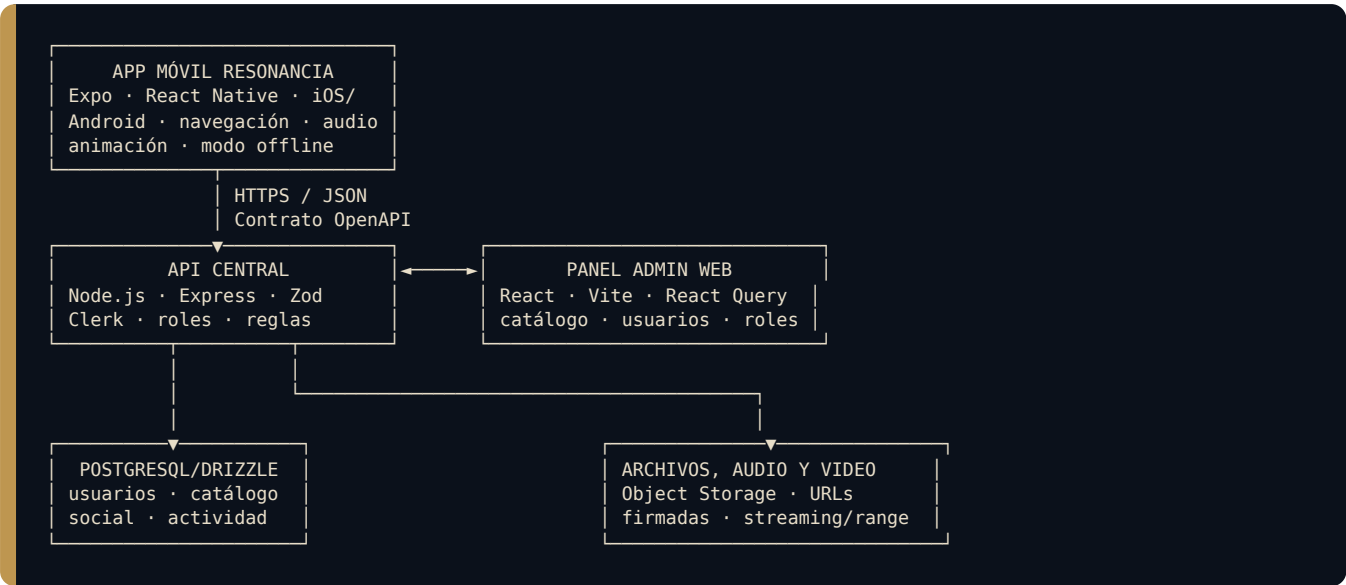
Conclusión ejecutiva

La plataforma tiene cimientos técnicos sólidos para un producto móvil con audio avanzado, contenido administrable y experiencias visuales complejas. Sus principales fortalezas son la separación entre cliente, API y datos; el contrato OpenAPI compartido; la especialización del motor de audio; y la existencia de control de acceso real en servidor.

Los principales puntos de atención son la cantidad de contextos globales de la app móvil, el tamaño de algunos coordinadores centrales, la dependencia de módulos nativos que exigen recompilar el cliente y el uso de sincronización de esquema con Drizzle Push en lugar de un historial de migraciones versionadas.

Este documento describe la arquitectura observada en el repositorio. No sustituye una auditoría de seguridad, rendimiento o cumplimiento.

2. Vista general del sistema



Componentes desplegados

Componente	Responsabilidad principal	Tecnología
Aplicación móvil	Experiencia de usuario, reproducción, mezcla, animación, modo offline	Expo, React Native, TypeScript
API Server	Reglas de negocio, identidad, autorización, catálogo, comunidad y almacenamiento	Node.js, Express, Zod
Base de datos	Persistencia transaccional y relaciones de producto	PostgreSQL, Drizzle ORM
Panel administrativo	Operación del catálogo, usuarios, roles, moderación y contenido	React, Vite, React Query
Contratos compartidos	Especificación de endpoints, tipos, hooks y validación	OpenAPI, Orval, Zod
Servicios externos	Identidad, compras, objetos, sesiones en vivo y distribución multimedia	Clerk, RevenueCat, Object Storage, Daily, CDN

3. Lenguajes, frameworks y herramientas

Lenguajes principales

Lenguaje/formato	Uso
TypeScript 5.9	App móvil, backend, panel administrativo y bibliotecas compartidas
JavaScript/ESM	Scripts de compilación y operación
SQL, expresado con Drizzle	Modelo y consultas PostgreSQL
YAML	Contrato OpenAPI y configuración
JSON	Configuración Expo, EAS y metadatos
Swift/Objective-C y Kotlin/Java	Dependencias y proyectos nativos administrados por Expo/React Native
Markdown	Documentación técnica y decisiones

Stack móvil verificado

- Expo SDK 54.
- React 19.1.
- React Native 0.81.5.
- Expo Router 6 con rutas tipadas.
- React Native Reanimated 4 y Worklets.
- React Native Gesture Handler.
- React Native SVG.
- TanStack React Query.
- AsyncStorage y SecureStore.
- Clerk Expo.
- Expo Audio y React Native Audio API.
- Expo Video, notificaciones, archivos, imágenes y compartir.
- RevenueCat para compras móviles.
- Daily React Native para sesiones en vivo.

Stack de servicios verificado

- Node.js y módulos ES.
- Express 5.
- PostgreSQL.
- Drizzle ORM y Drizzle Kit.
- Zod y drizzle-zod.
- OpenAPI 3.1.

- Orval para generación de clientes y esquemas.
- Pino para registros del servidor.
- esbuild para el bundle del backend.
- Vitest y Supertest para pruebas de rutas.

4. Organización del monorepo

El proyecto usa **pnpm workspaces**. Esta decisión permite mantener varios productos ejecutables y bibliotecas compartidas en un solo repositorio, con una única resolución de dependencias y comprobación de tipos coordinada.

```

/
├── artifacts/
│   ├── mobile/           App Expo/React Native
│   ├── api-server/       API Node/Express
│   ├── resonancia-admin/ Panel administrativo web
│   └── ...               Presentaciones y artefactos editoriales
├── lib/
│   ├── api-spec/         OpenAPI y configuración de Orval
│   ├── api-client-react/ Cliente React Query generado
│   ├── api-zod/          Esquemas Zod generados
│   ├── db/               Conexión, tablas y seeds PostgreSQL
│   └── premium/          Contexto compartido de premium
├── scripts/              Automatizaciones del repositorio
├── package.json
├── pnpm-workspace.yaml   Paquetes y catálogo de versiones
└── tsconfig.base.json    Base TypeScript compartida
  
```

Principio de dependencia

```

OpenAPI —genera→ cliente React Query —usa→ móvil/admin
    |
    └── genera→ esquemas Zod —usa→ API

DB schema —→ Drizzle ORM —usa→ API
API —→ PostgreSQL / Storage / servicios externos
  
```

La especificación OpenAPI es una pieza central. Un cambio de contrato debe propagarse mediante codegen antes de considerarse completo.

5. Arquitectura de la aplicación móvil

5.1 Navegación

La navegación usa **Expo Router** y su modelo basado en archivos. La carpeta `app/` contiene rutas normales, rutas dinámicas, grupos de autenticación y pestañas, además de modales de pantalla completa.

En la fecha de corte se contabilizan **123 archivos de pantalla/ruta**, sin contar layouts ni la pantalla de ruta no encontrada. Esto no significa 123 secciones visibles en el menú: muchas son detalles dinámicos, flujos, overlays y pantallas de soporte.

Los layouts principales organizan:

- autenticación;
- pestañas principales;
- stack raíz;
- modales y reproductores a pantalla completa;
- transiciones laterales, verticales y de desvanecimiento.

5.2 Composición de providers

El layout raíz compone servicios transversales antes de renderizar las pantallas. Entre ellos:

- autenticación y perfil;
- React Query;
- catálogo y sonidos;
- premium;
- reproductores;
- mezclador;
- notificaciones;
- hitos y rachas;
- bibliotecas, playlists y carpetas;
- videos;
- diario;
- tema visual y escenas;
- overlays globales.

El patrón dominante de estado es **React Context + hooks especializados**. React Query se utiliza para estado remoto y caché de API; los contextos coordinan estado de experiencia, audio y persistencia local.

5.3 Capas internas

Capa	Contenido típico
Rutas	Pantallas y navegación basada en archivos
Componentes	Tarjetas, reproductores, modales, fondos y controles
Contextos	Estado global, coordinación y efectos de ciclo de vida
Hooks	Consultas, sincronización, reproducción y comportamiento reutilizable
Datos/configuración	Catálogo incluido en la app, mapas de audio y metadatos
Librerías	Sincronización cloud, audio especializado, API y utilidades
Assets	Imágenes, fuentes y audio incluido en el bundle

5.4 Modelo offline-first

La app puede operar sin cuenta y conserva estado local con AsyncStorage. Cuando existe identidad Clerk, sincroniza actividad seleccionada con el backend.

El modelo distingue:

- **eventos append-only**, unidos y deduplicados;
- **favoritos y progreso**, unidos en la primera sincronización y luego tratados con autoridad local;
- **biblioteca ampliada**, que incluye presets y creaciones;
- historial que puede mantenerse local según el dominio.

Esta arquitectura favorece continuidad de uso sin conexión, pero obliga a definir cuidadosamente las reglas de borrado y resolución de conflictos entre dispositivos.

6. API y reglas de negocio

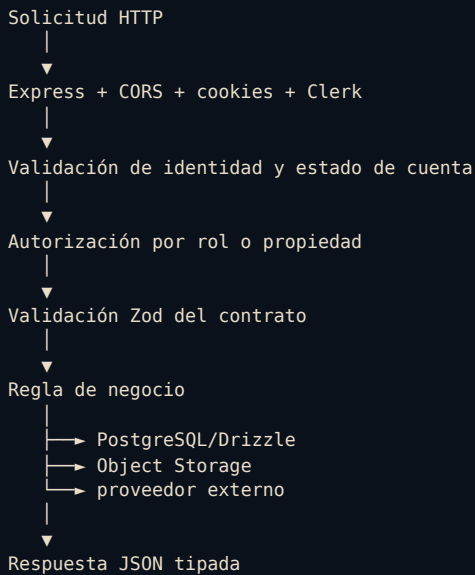
6.1 Runtime

La API es un servicio TypeScript/Node compilado con esbuild y ejecutado como módulo ES. Express monta todos los dominios bajo `/api`.

Dominios verificados:

- salud del servicio;
- usuarios y perfiles;
- amistades, follows y mensajería;
- notificaciones y push;
- catálogo, sonidos, videos y playlists;
- actividad, favoritos, progreso e hitos;
- mezclas y Geometrix compartidos;
- almacenamiento y cargas;
- postulaciones y perfiles profesionales;
- sesiones en vivo y calendario;
- administración y moderación.

6.2 Pipeline de solicitud



6.3 Contrato API

`lib/api-spec/openapi.yaml` es la definición canónica. Orval genera dos productos:

1. **Cliente React Query** para el móvil y el panel.
2. **Esquemas Zod** para validación en el servidor.

Beneficios:

- menos tipos duplicados y consultas consistentes;
- validación alineada con la documentación;
- divergencias detectables mediante typecheck y codegen.

Cuando cambia el contrato, la especificación y sus artefactos generados deben actualizarse juntos.

7. Base de datos

7.1 Tecnología

La persistencia central utiliza **PostgreSQL** con el driver `pg` y Drizzle ORM. La conexión requiere `DATABASE_URL` y se administra mediante un pool configurable.

7.2 Dominios de datos

El esquema está dividido en módulos. Agrupados por función:

Dominio	Entidades representativas
Identidad	usuarios, roles, perfiles resonadores/expansores
Catálogo	categorías, sesiones, audio, videos, sonidos, playlists, etiquetas
Actividad	favoritos, progreso, historial, hitos, biblioteca
Comunidad	mezclas, glifos, likes, comentarios, reportes, actividad
Social	amistades, follows, mensajes directos y likes
Operación	notificaciones, tokens push, cargas, eliminaciones de cuenta
Experiencias	escenas, ajustes Geometrix, descanso, sesiones en vivo y calendario
Admisión	postulaciones y configuraciones de guías

7.3 Evolución del esquema

El paquete de base de datos expone comandos `drizzle-kit push` y `push-force`. No se identificó un directorio de migraciones SQL versionadas en el estado revisado.

Implicación ejecutiva: el modelo actual es ágil para desarrollo, pero conviene incorporar migraciones versionadas, revisión y procedimiento de despliegue antes de aumentar el número de entornos o colaboradores.

7.4 Consistencia y concurrencia

En dominios sociales se utilizan operaciones transaccionales y bloqueos de fila para contadores derivados como likes. Los eventos de actividad emplean identificadores de cliente para deduplicación. Las relaciones y autorizaciones no se delegan al cliente: se validan en la API.

8. Identidad, roles y control de acceso

8.1 Proveedor de identidad

Clerk administra la identidad. En móvil, la cuenta es compatible con una experiencia local: el usuario puede iniciar con estado local y vincular sincronización cuando inicia sesión.

El backend instala middleware Clerk y transforma la identidad externa en un usuario interno de PostgreSQL.

8.2 Roles

El modelo contempla roles como:

- usuario;
- creador;
- administrador;
- moderador;

- expansor;
- resonador.

La autorización real vive en el servidor. El panel puede ocultar o mostrar acciones por experiencia de usuario, pero los endpoints protegidos exigen autenticación y rol.

8.3 Autorización de objetos

Las rutas de almacenamiento privado combinan:

- identidad;
- propiedad o concesión explícita;
- estado del recurso;
- rol editorial cuando corresponde.

Una referencia a una ruta privada no concede acceso por sí sola.

9. Almacenamiento y distribución multimedia

9.1 Objetos

La API se integra con almacenamiento de objetos mediante credenciales administradas por la plataforma. Se separan rutas públicas y privadas.

El flujo de carga general es:

```
Cliente autenticado
| solicita permiso + metadatos
▼
API valida MIME/tamaño/rol
| registra la intención
▼
URL PUT firmada y temporal
|
▼
Cliente sube directamente al storage
|
▼
API guarda/referencia el objectPath
```

El almacenamiento directo evita que archivos grandes atraviesen el proceso Node completo.

9.2 Audio y video

- Parte del audio esencial puede estar incluida en el bundle.
- El catálogo remoto utiliza URLs u object paths resueltos por la app.
- Los videos usan streaming y solicitudes por rango para permitir seek.
- La distribución por CDN puede configurarse para catálogos de video.

- Las imágenes usan caché de memoria y disco en la app.

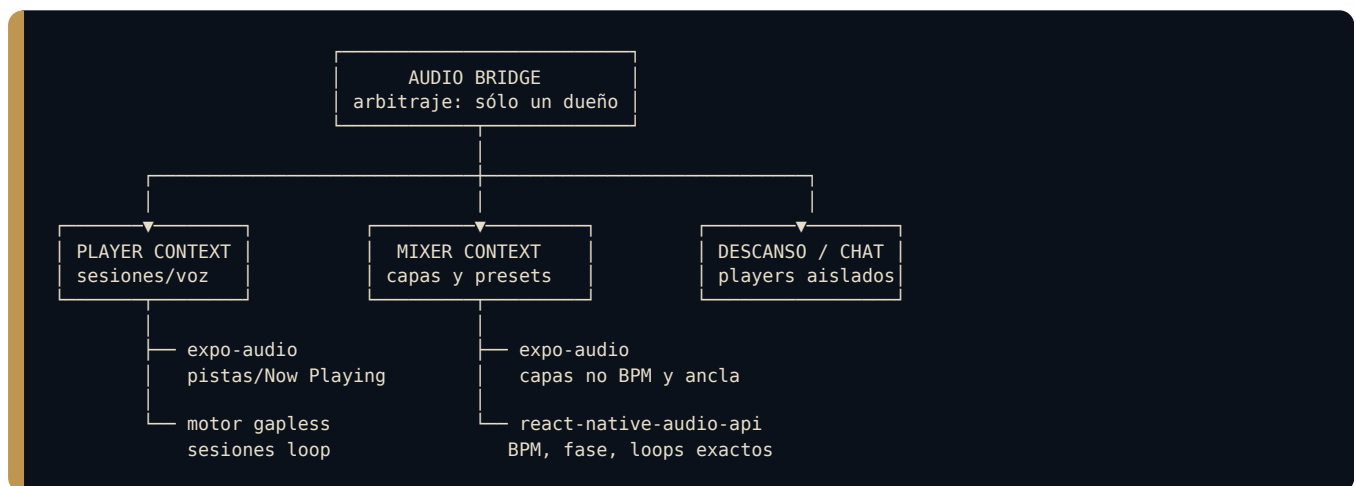
9.3 Formato de audio

La decisión de distribución documentada es **AAC/M4A** para producto, con bitrate distinto según música, voz o loops. WAV permanece como formato de master o fuente, no como formato preferido de distribución.

10. Arquitectura de audio

El audio es uno de los cimientos más especializados de RESONANCIA. La app no usa un solo player global para todo, sino un conjunto de motores coordinados.

10.1 Mapa de motores



10.2 Expo Audio

expo-audio es el dueño de la sesión de audio del sistema. Se utiliza para:

- reproducción principal;
- voz y capas complementarias;
- metadatos de pantalla bloqueada;
- controles remotos;
- reproducción en modo silencio;
- continuidad en segundo plano;
- ancla del sistema cuando otro motor produce el audio audible.

Los players principales son persistentes para evitar recreación innecesaria al cambiar de sesión.

10.3 Motor nativo gapless/BPM

react-native-audio-api aporta un grafo de audio nativo similar a Web Audio. Se utiliza cuando se necesita:

- loop sin separación audible;

- sincronización por BPM;
- capas bloqueadas en fase;
- `LoopEnd` musical calculado;
- ganancia maestra;
- pausa/reanudación conservando fase;
- reproducción de loops de sesión.

El motor calcula límites de loop usando tempo, compases y duración de buffer. Para no competir por la sesión iOS, desactiva su administración de sesión y deja esa responsabilidad a Expo Audio.

10.4 Mezclador

El mezclador administra:

- sonidos activos;
- volumen por capa y volumen maestro;
- presets;
- filtros y estados persistidos;
- sonidos locales y remotos;
- binaurales;
- offsets de fase;
- metadata de pantalla bloqueada;
- temporizador de sueño;
- pausa y reanudación conjunta.

Los sonidos BPM pasan por el motor nativo. Los sonidos no BPM pueden usar Expo Audio, según su naturaleza.

10.5 AudioBridge

`audioBridge` es un registro central de funciones de detención. Antes de comenzar una experiencia, cada subsistema detiene a sus competidores.

Esto evita situaciones como:

- una sesión y una mezcla sonando simultáneamente;
- chat reproduciéndose sobre una meditación;
- audio de descanso persistiendo al cambiar de experiencia.

10.6 Segundo plano, bloqueo y temporizadores

La configuración nativa habilita audio en background. Los temporizadores no dependen únicamente de intervalos JavaScript, porque el sistema puede congelarlos con la pantalla bloqueada.

La arquitectura combina:

- timestamp absoluto de expiración;
- listener de estado nativo;
- recuperación mediante AppState;

- intervalo JavaScript sólo para refrescar la interfaz;
- ancla Expo Audio para controles del sistema.

10.7 Riesgos operativos del audio

- Los módulos de audio nativo requieren recompilar el dev client.
- Expo Audio debe seguir siendo el único propietario de AVAudioSession en iOS.
- Los cambios de rutas BPM, offsets o loops deben mantenerse coordinados.
- El motor requiere pruebas en dispositivos físicos; el preview web no representa el comportamiento nativo.

11. Arquitectura de animación y Geometrix

11.1 Tecnologías

La capa visual avanzada se construye con:

- React Native Reanimated 4;
- Worklets ejecutados en el hilo de UI;
- Gesture Handler;
- React Native SVG;
- gradientes, blur y efectos nativos de Expo;
- ViewShot para capturas;
- persistencia de recetas en lugar de imágenes rasterizadas.

11.2 Geometrix

Geometrix es un editor de composiciones. Una creación es una **receta de datos**:

- capas activas;
- orden;
- geometría base;
- transformaciones;
- color y opacidad;
- animaciones;
- ajustes maestros;
- visibilidad y estado del lienzo.

La receta puede renderizarse de nuevo en distintos contextos, guardarse localmente, sincronizarse y compartirse.

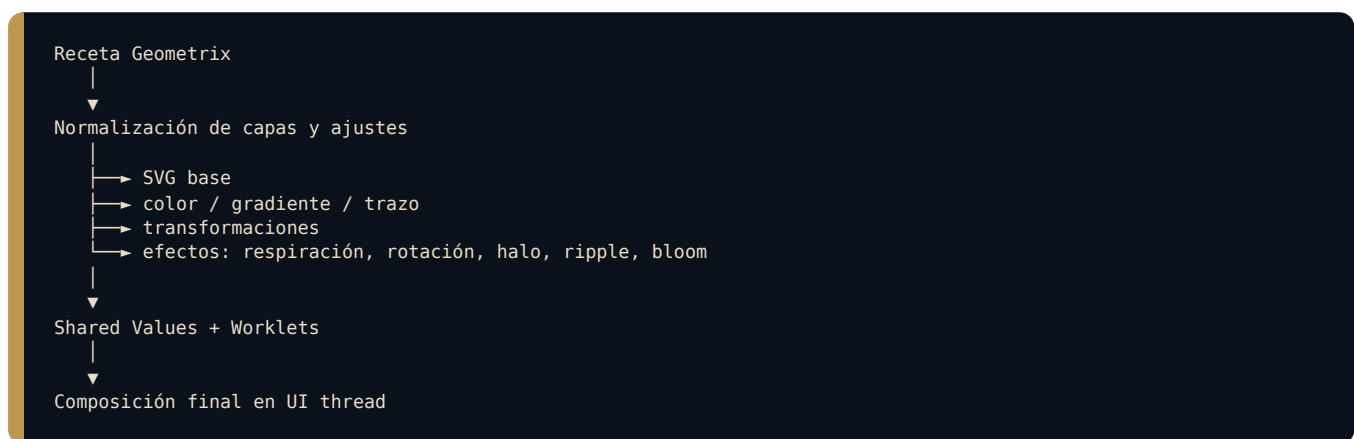
11.3 Separación editor/renderizador

La arquitectura diferencia:

- el editor completo, con gestos, carrusel, historial y herramientas;
- el renderizador de escenas, usado en Inicio y previews;
- las tarjetas y modales de selección;
- el modelo persistido.

Esta separación permite usar una creación como ambientación sin montar todo el editor.

11.4 Pipeline visual



11.5 Principios de rendimiento

- No enviar estado React por cada frame de un gesto.
- Mantener animaciones de alta frecuencia en UI thread.
- Pausar o desmontar escenas fuera de foco.
- Cancelar loops al desmontar o cambiar de escena.
- Reducir copias decorativas en superficies de uso prolongado.
- Mantener el DOM/árbol estable en drag-reorder y animar transforms.
- Usar SVG vectorial a tamaño real para zoom nítido.

El renderizador dispone de una calidad optimizada para Inicio que conserva las geometrías base y reduce duplicaciones decorativas. El editor mantiene la calidad completa.

12. Panel administrativo

El panel administrativo es una aplicación web independiente montada bajo `/admin/`.

Stack

- React y TypeScript.
- Vite.

- React Query.
- Wouter para routing.
- componentes de interfaz reutilizables.
- Clerk con sesión same-origin.
- cliente generado desde OpenAPI.

Responsabilidades

- dashboard operativo;
- usuarios y roles;
- catálogo y categorías;
- carga y edición de audio, imágenes y video;
- playlists y escenas;
- moderación de contenido compartido;
- postulaciones y perfiles profesionales;
- sesiones en vivo y configuraciones.

La interfaz es una herramienta operativa; la autorización efectiva permanece en la API.

13. Integraciones externas

Integración	Uso arquitectónico
Clerk	identidad, sesión y vínculo con usuario interno
RevenueCat	estado premium y compras in-app
Object Storage	imágenes, audio, adjuntos y videos
Daily	infraestructura de sesiones en vivo/WebRTC
Expo/EAS	desarrollo, dev client y builds móviles
Servicios push	registro de tokens y envío de notificaciones
CDN de video	distribución de catálogo audiovisual cuando está configurada

Las claves y credenciales se administran como secretos del entorno y no deben incorporarse al código, documentación o builds.

14. Compilación, ejecución y operación

Comandos principales

Objetivo	Comando
API en desarrollo	<code>pnpm --filter @workspace/api-server run dev</code>
Admin en desarrollo	<code>pnpm --filter @workspace/resonancia-admin run dev</code>
App móvil	workflow Expo del artifact móvil
Typecheck general	<code>pnpm run typecheck</code>
Build general	<code>pnpm run build</code>
Regenerar API	<code>pnpm --filter @workspace/api-spec run codegen</code>
Actualizar esquema dev	<code>pnpm --filter @workspace/db run push</code>

Build móvil

El proyecto contiene configuración Expo/EAS y proyectos nativos. La New Architecture de React Native está habilitada.

Cambios en estos componentes requieren un nuevo dev client o build:

- audio nativo;
- WebRTC/Daily;
- compras;
- plugins Expo;
- permisos;
- configuración iOS/Android.

Un cambio puramente TypeScript/JS puede probarse con Metro, siempre que no altere módulos nativos.

Observabilidad y validación

- TypeScript para comprobación estática.
 - Vitest/Supertest en rutas de backend.
 - Pino para logs estructurados.
 - pruebas de capacidad y preflight editorial en scripts dedicados.
 - comprobación real de audio y background en dispositivos.
-

15. Flujos críticos de extremo a extremo

15.1 Abrir y reproducir una sesión

```
Pantalla de catálogo
→ resuelve metadata local/remota
→ verifica acceso premium
→ detiene motores competidores
→ configura sesión de audio
→ selecciona Expo Audio o motor loop
→ publica Now Playing
→ registra progreso/actividad
→ sincroniza cuando hay cuenta y red
```

15.2 Editar catálogo desde administración

```
Administrador autenticado
→ panel usa cliente generado
→ API valida sesión y rol
→ Zod valida payload
→ Drizzle escribe PostgreSQL
→ archivos van por URL firmada
→ catálogo publicado aparece en GET /catalog
→ móvil hidrata datos remotos sobre el modelo local
```

15.3 Crear y compartir Geometrix

```
Editor compone receta
→ guarda localmente
→ biblioteca cloud sincroniza si corresponde
→ compartir publica JSON de receta
→ comunidad renderiza preview en cliente
→ likes/comentarios pasan por API y DB
```

16. Evaluación de madurez arquitectónica

Área	Estado	Lectura ejecutiva
Separación cliente/API/datos	Sólido	Límites claros y servicios independientes
Contratos API	Sólido	OpenAPI + codegen reduce divergencias
Identidad y roles	Sólido	Enforcement en servidor y roles persistidos
Audio especializado	Avanzado	Motores diferenciados y coordinación explícita
Animación/Geometrix	Avanzado	UI thread, SVG y recetas persistentes
Operación de catálogo	Sólido	Admin conectado a la misma API y DB
Offline/sincronización	En evolución	Buen modelo base; conflictos/borrados requieren disciplina
Evolución DB	A reforzar	Falta historial de migraciones versionadas
Modularidad móvil	A reforzar	Muchos providers y coordinadores centrales grandes
Observabilidad	En evolución	Logs y pruebas disponibles; falta una visión integral de producto

Prioridades técnicas recomendadas

1. Adoptar migraciones de base de datos versionadas y un procedimiento de despliegue.
2. Mantener OpenAPI como punto único y obligatorio para todo cambio de contrato.
3. Dividir gradualmente los contextos móviles de mayor tamaño por responsabilidad.
4. Ampliar pruebas automatizadas de sincronización y del motor de audio.
5. Medir rendimiento en dispositivos físicos con escenarios prolongados.
6. Formalizar métricas, alertas y trazabilidad entre cliente y API.

17. Guía de incorporación para programadores

Primer día

1. Leer `replit.md` y este informe.
2. Revisar `pnpm-workspace.yaml` y el `package.json` raíz.
3. Identificar el componente asignado: móvil, API, admin o biblioteca.
4. Ejecutar typecheck antes de cambiar código.

Para trabajar en móvil

1. Comenzar por `artifacts/mobile/app/_layout.tsx`.
2. Seguir la ruta específica desde `app/`.
3. Identificar qué contexto posee el estado.
4. Verificar si el cambio usa módulo nativo.
5. Probar pérdida de foco, background y cleanup cuando exista audio/animación.

Para trabajar en API

1. Localizar el router en `artifacts/api-server/src/routes/`.
2. Verificar contrato en OpenAPI.
3. Revisar middleware de identidad/rol.
4. Revisar esquema Drizzle del dominio.
5. Regenerar cliente y Zod cuando cambie el contrato.
6. Añadir pruebas de ruta.

Para trabajar en datos

1. No modificar tablas sin revisar consumidores de API y móvil.
2. Probar relaciones, índices y concurrencia.
3. Coordinar cambios de esquema con el procedimiento del entorno.
4. Evitar cambios destructivos sin migración y respaldo.

Para trabajar en audio o Geometrix

1. Leer los comentarios de arquitectura del motor correspondiente.
 2. Respetar el propietario de la sesión de audio.
 3. No introducir actualizaciones React por frame.
 4. Cancelar listeners, timers, animations y nodos al cerrar.
 5. Probar en dispositivo físico y con pantalla bloqueada cuando aplique.
-

18. Mapa de archivos esenciales

Área	Punto de entrada
Layout móvil	<code>artifacts/mobile/app/_layout.tsx</code>
Tabs	<code>artifacts/mobile/app/(tabs)/_layout.tsx</code>
Reproductor	<code>artifacts/mobile/context/PlayerContext.tsx</code>
Mezclador	<code>artifacts/mobile/context/MixerContext.tsx</code>
Arbitraje de audio	<code>artifacts/mobile/context/audioBridge.ts</code>
Motor gapless	<code>artifacts/mobile/lib/bpmAudioEngine.ts</code>
Render de escenas	<code>artifacts/mobile/components/SceneAnimationInline.tsx</code>
Editor Geometrix	<code>artifacts/mobile/app/(tabs)/geometrix.tsx</code>
Sincronización	<code>artifacts/mobile/lib/cloudSync.ts</code>
API	<code>artifacts/api-server/src/app.ts</code>
Routers	<code>artifacts/api-server/src/routes/index.ts</code>
Autorización	<code>artifacts/api-server/src/middlewares/</code>
OpenAPI	<code>lib/api-spec/openapi.yaml</code>
Codegen	<code>lib/api-spec/orval.config.ts</code>
Base de datos	<code>lib/db/src/index.ts</code>
Esquemas DB	<code>lib/db/src/schema/</code>
Panel admin	<code>artifacts/resonancia-admin/src/App.tsx</code>

19. Cierre

RESONANCIA no es únicamente una aplicación de contenido. Es una plataforma con motores multimedia, edición visual, comunidad, operación editorial, identidad, sincronización y servicios compartidos.

Para preservar sus cimientos, cualquier evolución debería mantener cuatro reglas:

1. **El contrato de API es explícito y generado.**
2. **La autorización se decide en el servidor.**
3. **El audio y la animación deben respetar el ciclo de vida nativo.**
4. **Los datos persistentes necesitan cambios versionados y reversibles.**

Con estas reglas, la arquitectura actual puede seguir creciendo sin perder claridad, estabilidad ni capacidad de incorporación de nuevos programadores.

Fuentes técnicas revisadas

- Configuración del workspace y comandos raíz.
- Manifiestos de la app móvil, API, DB y panel administrativo.
- Layout raíz y contextos de reproducción/mezcla.
- Motor de audio gapless/BPM y puente de arbitraje.
- Renderizador de escenas y editor Geometrix.
- Contrato OpenAPI y configuración de Orval.
- Esquemas Drizzle y routers Express.
- Configuración Expo, EAS, iOS y Android.
- Documentación operativa del repositorio.

Fin del informe